The Journal of Open Source Software

k-diagram: Rethinking Forecasting Uncertainty via Polar-based Visualization

Kouao Laurent Kouadio [1,2]

1 School of Geosciences and Info-Physics, Central South University, Changsha, Hunan, 410083, China **2** Hunan Key Laboratory of Nonferrous Resources and Geological Hazards Exploration, Changsha, Hunan, 410083, China

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 09 May 2025
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Summary

`k-diagram` is an open-source Python package providing specialized diagnostic plots based on polar coordinates for the comprehensive evaluation of forecasting models, with a strong emphasis on understanding and visualizing predictive uncertainty. It offers a suite of functions ('k-diagrams') to assess prediction interval coverage, consistency, anomaly magnitude, model drift across horizons, feature importance patterns, and standard evaluation metrics (e.g., via Taylor diagrams). Designed to complement traditional evaluation methods, `k-diagram` aids interpretability and facilitates deeper insights into forecast reliability, particularly for complex spatiotemporal systems relevant to environmental science and geohazard assessment.

Statement of Need

Evaluating the performance of forecasting models, especially in high-stakes domains like environmental hazard prediction (e.g., land subsidence, flooding), requires moving beyond simple point-forecast accuracy metrics. Understanding the structure, reliability, and limitations of a model's predictive uncertainty is critical for risk assessment and informed decision-making (Murphy & Winkler, 1993). However, standard visualization tools often struggle to present multi-faceted uncertainty information (e.g., interval width, coverage success/failure, temporal drift) simultaneously and intuitively across many data points or locations.

Existing libraries provide excellent general plotting (e.g., `matplotlib` (Hunter, 2007), `seaborn` (Waskom, 2021)) and some model evaluation metrics (e.g., `scikit-learn` (Pedregosa et al., 2011)), but lack dedicated tools for the specific diagnostic visualization of uncertainty characteristics in a way that highlights patterns related to temporal progression or spatial distribution inherent in many forecasting problems. This gap became evident during research applying advanced deep learning models to forecast complex phenomena like urban land subsidence in challenging environments such as Nansha, China (Liu et al., 2024). While models could generate probabilistic forecasts, interpreting the spatiotemporal behavior of the predicted uncertainty remained difficult.

`k-diagram` addresses this need by introducing a suite of visualizations primarily based on polar coordinates. This approach allows mapping different dimensions of forecast performance and uncertainty onto angle and radius, providing compact overviews and revealing patterns potentially obscured in Cartesian plots. For example, mapping time or sample index to angle allows visual inspection of consistency, drift, or anomaly clustering in a circular layout. `k-diagram` aims to make the analysis of predictive uncertainty more intuitive and actionable, treating uncertainty itself as a crucial signal to be diagnosed.

Functionality

k-diagram is implemented in Python ??, leveraging core scientific libraries including numpy (Harris et al., 2020), pandas (McKinney, 2010; Reback et al., 2020), matplotlib (Hunter, 2007), scipy (Virtanen et al., 2020), and scikit-learn (Pedregosa et al., 2011). Its main features include:

- **Uncertainty Diagnostics:** A collection of polar plots under `kdiagram.plot.uncertainty` for visualizing:
  - Prediction interval coverage (point-wise diagnostics and overall scores).
  - Anomaly magnitude (severity and type of interval failures).
  - Interval width (magnitude and consistency over time/samples).
  - Model drift (average uncertainty increase over forecast horizons).
  - Uncertainty drift (evolution of uncertainty patterns using concentric rings).
  - Prediction velocity (rate of change of central forecasts).
- **Model Evaluation:** Functions under `kdiagram.plot.evaluation` for generating standard and enhanced Taylor Diagrams (Taylor, 2001) to compare models based on correlation, standard deviation, and RMSD relative to a reference.
- **Feature Importance:** Radar charts under `kdiagram.plot.feature_based` for visualizing and comparing feature importance profiles ("fingerprints") across different models or contexts.
- **Relationship Visualization:** Polar scatter plots under `kdiagram.plot.relationship` mapping true values to angle and (normalized) predictions to radius.
- **Data Utilities:** Helper functions under `kdiagram.utils` for detecting, validating, and reshaping pandas DataFrames containing quantile-based forecast data, facilitating data preparation for the plotting functions.
- **Command-Line Interface (CLI):** A `k-diagram` command allowing users to generate key plots directly from CSV files without writing Python scripts.
- **Customization:** Plots offer various parameters for adjusting appearance, angular coverage (`acov`), colormaps, normalization, and labeling.

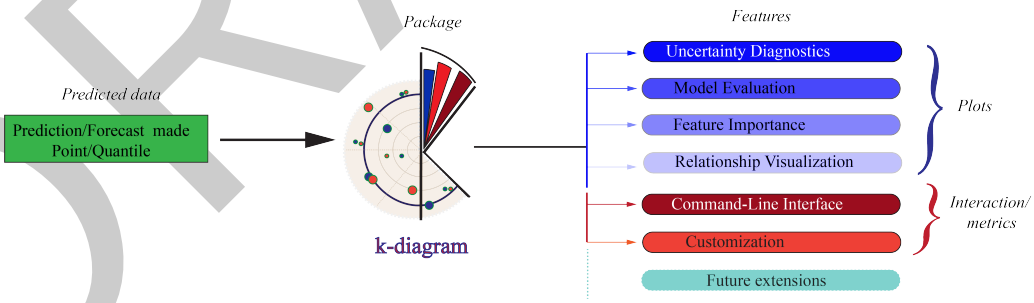

Figure 1: Structure of the k-diagram, illustrating the identification of global features.

Related Work

The visualization methods implemented in `k-diagram` were developed alongside research applying deep learning models to complex environmental forecasting challenges. Specifically, these polar diagnostics were utilized to analyze and interpret the uncertainty associated with land subsidence predictions using an Extreme Temporal Fusion Transformer model in Zhongshan city, China. The results and application context of that specific study are detailed in a separate manuscript submitted to the International Journal of Forecasting (Kouadio et al., 2025). This JOSS paper focuses specifically on the `k-diagram` software package itself – its design, functionality, and general applicability for visualizing forecast uncertainty.

DRAFT

Installation

The latest stable release of `k-diagram` is available on the Python Package Index (PyPI) and can be installed using pip:

```
pip install k-diagram
```

Detailed installation instructions are available in the documentation.

Documentation

Comprehensive documentation, including a User Guide explaining the concepts behind the plots, a gallery of examples, and a detailed API reference, is hosted on ReadTheDocs: https://k-diagram.readthedocs.io/.

License

`k-diagram` is made available under the OSI-approved Apache License 2.0. The full license text can be found in the `LICENSE` file in the root of the repository.

Example Usage

The following snippet Figure 2 demonstrates generating an Anomaly Magnitude plot, identifying where actual values fall outside the 10th-90th percentile prediction interval.

```python
import kdiagram as kd
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# 1. Generate Sample Data
np.random.seed(42)
n_points = 180
df = pd.DataFrame({'sample_id': range(n_points)})
df['actual'] = np.random.normal(loc=20, scale=5, size=n_points)
df['q10'] = df['actual'] - np.random.uniform(2, 6, size=n_points)
df['q90'] = df['actual'] + np.random.uniform(2, 6, size=n_points)
# Add anomalies
under_indices = np.random.choice(n_points, 20, replace=False)
df.loc[under_indices, 'actual'] = df.loc[under_indices, 'q10'] - \
                                  np.random.uniform(1, 5, size=20)
available = list(set(range(n_points)) - set(under_indices))
over_indices = np.random.choice(available, 20, replace=False)
df.loc[over_indices, 'actual'] = df.loc[over_indices, 'q90'] + \
                                 np.random.uniform(1, 5, size=20)

# 2. Create the Anomaly Magnitude Plot
ax = kd.plot_anomaly_magnitude(
    df=df,
    actual_col='actual',
    q_cols=['q10', 'q90'], # Provide lower and upper bounds as a list
    title="Sample Anomaly Magnitude Plot",
    cbar=True,            # Show color bar indicating magnitude
    s=100,                # maker size adjusted
    verbose=0             # Suppress console summary for brevity

)
```

DRAFT

```
# In a script, use plt.show() or ax.figure.savefig("plot.png")
# plt.show() # Not needed if saving or in some environments
```

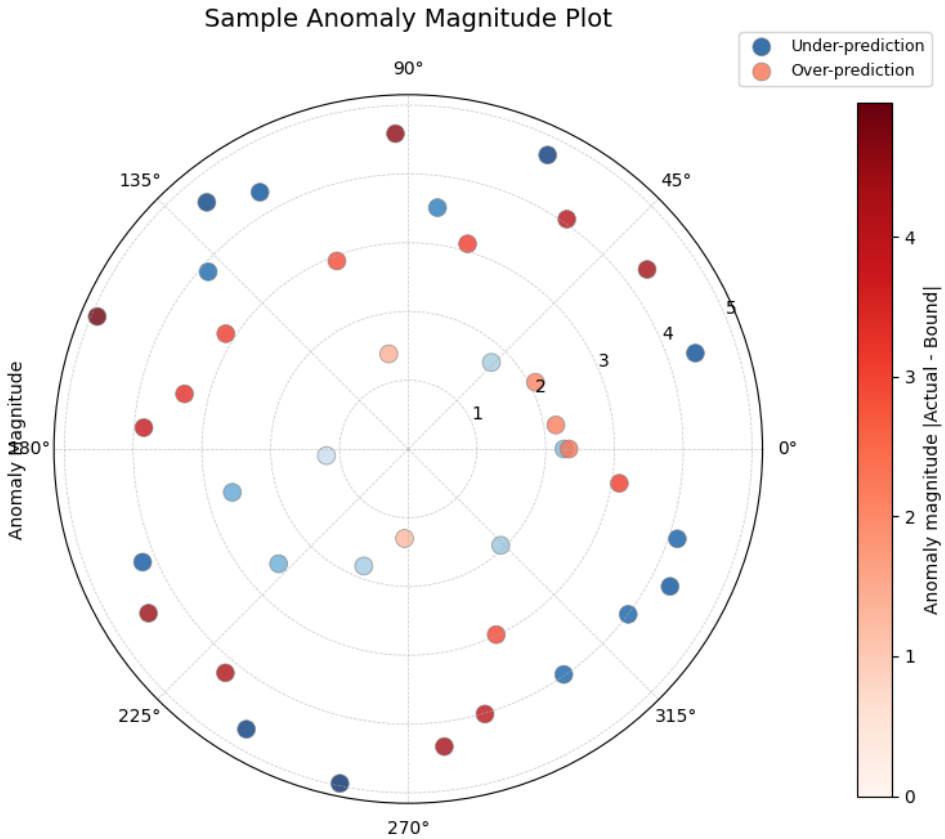

Figure 2: Example k-diagram plot demonstrating [Specific Feature Shown, e.g., Anomaly Magnitude] for simulated data. Red points indicate overpredictions, blue points under-predictions,relative to the interval bounds. Radius indicates the magnitude of the anomaly.

Contributing

Contributions to `k-diagram` are highly encouraged! We welcome bug reports, feature requests, documentation improvements, and code contributions. Please refer to the `CONTRIBUTING.rst` guide in the documentation (or repository) for detailed guidelines on how to contribute. Development discussions and issue tracking occur on the GitHub repository.

Acknowledgements

We extend our sincere gratitude to the anonymous colleagues who provided invaluable feedback during the development of `k-diagram`, as well as those who rigorously tested its early iterations. Their insights and dedication were instrumental in refining the software.

We also appreciate the constructive feedback from early users and reviewers, whose contributions have significantly enhanced the quality and usability of the project.

References

Harris, C. R., Millman, K. J., Walt, S. J. van der, Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Memmesheimer, S., Backx, T., & others. (2020). Array programming with NumPy. *Nature*, *585*(7825), 357–362. https://doi.org/10.1038/s41586-020-2649-2

Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, *9*(3), 90–95. https://doi.org/10.1109/MCSE.2007.55

Kouadio, L., Liu, R., Loukou, K. G. H., Liu, J., & Liu, W. (2025). Understanding uncertainty in land subsidence forecasting. *International Journal of Forecasting*.

Liu, J., Liu, W., Allechy, F. B., Zheng, Z., Liu, R., & Kouadio, K. L. (2024). Machine learning-based techniques for land subsidence simulation in an urban area. *Journal of Environmental Management*, *352*(2024), 17. https://doi.org/https://doi.org/10.1016/j.jenvman.2024.120078

McKinney, W. (2010). *Data Structures for Statistical Computing in Python*. 56–61.

Murphy, A. H., & Winkler, R. L. (1993). What is a good forecast? Probability, surprise, consensus, information, and skill in the evaluation of forecasts. *Weather and Forecasting*, *8*(3), 287–303. https://doi.org/10.1175/1520-0434(1993)008%3C0287:WIAGFP%3E2.0.CO;2

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Olsczewski, B., Blondel, M., Prejbal, F., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, *12*, 2825–2830.

Reback, J., Bossche, J. V. den, McKinney, W., Reedy, T. J., Bergstrom, G., Augspurger, T., Brooke, M., Ciampi, S., Lee, A., Lustig, I., & others. (2020). Pandas 1.0, a fundamental rewrite of the library. *arXiv Preprint arXiv:2005.03268*.

Taylor, K. E. (2001). Summarizing quantitative measures of model performance in a single diagram. *Journal of Geophysical Research: Atmospheres*, *106*(D7), 7183–7192. https://doi.org/10.1029/2000JD900719

Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D., Burovski, E., Peterson, P., Wehrer, W., Huttunen, J., Håcklin, T., Straub, S., Pike, J., Petroff, A., Conti, K., Lieberman, R., Pollard, J., Nichols, B., Raimi, J., … Vázquez-Baeza, Y. (2020). SciPy 1.0: fundamental algorithms for scientific computing in Python. *Nature Methods*, *17*(3), 261–272. https://doi.org/10.1038/s41592-019-0686-2

Waskom, M. L. (2021). Seaborn: Statistical data visualization. *Journal of Open Source Software*, *6*(60), 3021. https://doi.org/10.21105/joss.03021

DRAFT